

String Manipulation

1. Reverse a string without using inbuilt reverse functions.

Simple Explanation: To reverse a string without using the built-in function, we can convert the string into a character array. Then, we loop through the array **backward** (from the last character to the first) and print or build the new string one character at a time.

```
public class StringReverser {
    public static String reverseString(String str) {
        if (str == null || str.isEmpty()) {
            return str;
        }

        // Convert the string to a character array
        char[] chars = str.toCharArray();
        String reversed = "";

        // Loop backward from the last character
        for (int i = chars.length - 1; i >= 0; i--) {
            reversed = reversed + chars[i];
        }
        return reversed;
    }
}
```

2. Check if a string is a palindrome.

Simple Explanation: A string is a palindrome if it reads the same forward and backward (like "MADAM"). We can check this by comparing the character at the start of the string with the character at the end. We move inwards from both sides until we meet in the middle. If any pair doesn't match, it's not a palindrome.

```
public class PalindromeChecker {
    public static boolean isPalindrome(String str) {
        if (str == null) {
            return false;
        }

        // Normalize the string (lowercase, remove spaces for better
        check)
        String cleanStr = str.toLowerCase().replaceAll("\\s+", "");
        int left = 0;
        int right = cleanStr.length() - 1;

        // Compare characters moving inward
        while (left < right) {
            if (cleanStr.charAt(left) != cleanStr.charAt(right)) {
```

```
        return false; // Found a mismatch
    }
    left++;
    right--;
}
return true; // All pairs matched
}
```

3. Find duplicate characters in a string.

Simple Explanation: To find duplicates, we can use a **HashMap** (or array/Set) to keep count of how many times each character appears. We loop through the string, and if the count for a character goes above one, we know it's a duplicate.

```
import java.util.HashMap;
import java.util.Map;
import java.util.Set;

public class DuplicateFinder {
    public static void findDuplicates(String str) {
        if (str == null || str.isEmpty()) {
            return;
        }

        // Map to store character counts
        Map<Character, Integer> charCounts = new HashMap<>();

        // Count frequency of each character
        for (char c : str.toLowerCase().toCharArray()) {
            if (c != ' ') { // Ignore spaces
                charCounts.put(c, charCounts.getOrDefault(c, 0) + 1);
            }
        }

        // Print characters with count > 1
        Set<Character> keys = charCounts.keySet();
        for (char key : keys) {
            if (charCounts.get(key) > 1) {
                System.out.println("Duplicate found: " + key);
            }
        }
    }
}
```

4. Remove all whitespaces from a string without replace().

Simple Explanation: Instead of using the built-in `replace()` method, we can loop through the string character by character. We check if the character is a space, and if it is **not** a space, we keep it and build a new string.

```
public class WhitespaceRemover {
    public static String removeSpaces(String str) {
        if (str == null) {
            return null;
        }

        StringBuilder result = new StringBuilder();

        // Loop through each character
        for (int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);

            // Check if the character is NOT a whitespace
            if (c != ' ' && c != '\t' && c != '\n') {
                result.append(c);
            }
        }
        return result.toString();
    }
}
```

5. Count vowels and consonants in a string.

Simple Explanation: We loop through the string and check each character. We define vowels ('a', 'e', 'i', 'o', 'u'). If a character is a letter and matches a vowel, we count it. If it's a letter but not a vowel, we count it as a consonant.

```
public class VowelConsonantCounter {
    public static void countVowelsAndConsonants(String str) {
        if (str == null || str.isEmpty()) {
            return;
        }

        int vowels = 0;
        int consonants = 0;

        // Convert to lowercase for easier comparison
        String cleanStr = str.toLowerCase();

        for (int i = 0; i < cleanStr.length(); i++) {
            char ch = cleanStr.charAt(i);

            // Check if character is a letter
            if (ch >= 'a' && ch <= 'z') {

                // Check for vowels
                if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' ||
ch == 'u') {
                    vowels++;
                } else {
                    // Everything else that is a letter is a consonant
                    consonants++;
                }
            }
        }
    }
}
```

```
        }  
    }  
    }  
    System.out.println("Vowels: " + vowels);  
    System.out.println("Consonants: " + consonants);  
}  
}
```

6. Check if a string contains only digits.

Simple Explanation: We check if a string contains only digits by looping through every character. For each character, we use the built-in `Character.isDigit()` method to check if it's a number (0-9). If we find even one character that is not a digit, we immediately know the check fails.

```
public class DigitChecker {  
    public static boolean containsOnlyDigits(String str) {  
        if (str == null || str.isEmpty()) {  
            return false;  
        }  
  
        // Loop through each character  
        for (int i = 0; i < str.length(); i++) {  
            // Check if the character is NOT a digit  
            if (!Character.isDigit(str.charAt(i))) {  
                return false;  
            }  
        }  
        return true; // All characters were digits  
    }  
}
```

7. Reverse each word in a sentence individually.

Simple Explanation: First, we split the sentence into individual words using the space character. Then, we loop through the list of words. For every word, we reverse its characters. Finally, we join the reversed words back together with spaces.

```
public class WordReverser {  
    public static String reverseEachWord(String sentence) {  
        if (sentence == null || sentence.isEmpty()) {  
            return sentence;  
        }  
  
        // Split the sentence into words  
        String[] words = sentence.split(" ");  
        StringBuilder reversedSentence = new StringBuilder();
```

```
        // Reverse each word individually
        for (String word : words) {
            StringBuilder reversedWord = new StringBuilder(word);
            reversedWord.reverse(); // Using the built-in reverse method
            // for simplicity in this step

            // Append the reversed word and a space
            reversedSentence.append(reversedWord).append(" ");
        }

        // Trim any trailing space and return
        return reversedSentence.toString().trim();
    }
}
```

8. Sort characters of a string alphabetically.

Simple Explanation: The easiest way to sort the characters is to convert the string into a character array. We then use the built-in **Arrays.sort()** method, which quickly sorts all elements of the array alphabetically. Finally, we convert the sorted array back into a string.

```
import java.util.Arrays;

public class CharacterSorter {
    public static String sortStringCharacters(String str) {
        if (str == null || str.isEmpty()) {
            return str;
        }

        // Convert the string to a character array
        char[] chars = str.toCharArray();

        // Use the built-in Arrays.sort method
        Arrays.sort(chars);

        // Convert the sorted array back to a string
        return new String(chars);
    }
}
```